Learning situation # 2 Monte Carlo & Confidence Intervals

1. **Fine Random Number Generators**. For the previous lab, it was simpler to use **rand()** for a first test (or more `precisely (double) rand()/(double) RAND_MAX` using `stdlib`. As said previously this function is not suitable for scientific work. You should now know that for scientific applications the default system random number generators are often statistically weak**. In the previous lab, we have said that such generators have to be banned for modern scientific programming**.

A – For simulations, it is far better to use a fast generator such as Mersenne Twister proposed by Matsumoto, particularly if compiled with optimization options. This generator is proposed by many language APIs and library (C++, C, Python,…). Always keep an eye on MT initialization (you can keep what Makoto proposes in his original version or even test the SFMT version. You can find and test the original source code here :

https://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/MT2002/emt19937ar.html

B – A generator proposed by Pierre L'Ecuyer (Université de Montréal) is named MRG32K3A. The code is provided on Moodle (along with two other high-quality but slower generators). Unlike Makoto's generator, this one does not suffer from the statistical weaknesses that make the former unsuitable for cryptographic applications. The scientific paper describing it can be found below :

https://pubsonline.informs.org/doi/abs/10.1287/opre.47.1.159

C – To test the performance of these two generators, compile without optimization options first, and then with them. Test 1 000 000 000 calls to the functions drawing pseudo-random numbers between 0 and 1. All code should be written in C (or Fortran). C++ is not needed here, since we do not deal with objects (we could).

2) Computing of confidence intervals around the simulated mean. The user inputs the number of replicates (experiments) he wants to perform before computing a confidence interval at 95% – **to do so, inspire yourself from the technique and table given in the appendix** of this lab for (α=0.05). See whether the number of replicates improves your results and decreases the confidence radius. Be careful of what you find on the Internet or with your favorite LLM, in scientific computing we rarely deal with the full statistical population! Stick to the method proposed in appendix. See the 2D OMP exemple computing PI, and keep this approach for a 3D computing with Monte Carlo sampling (test with the volume of a sphere 4/3 x PI x R^3).

The number of random drawings (sampling) for each individual replicate (each experiment) is a sensitive parameter. Be careful when you do this comparison. We have very few significant numbers in a float variable (only 7 significant decimal digits for `floats`), for scientific computing prefer `double` instead. It is preferred for scientific computing. When you have an estimation of the standard deviation, you can also see how to compute the 'standard error' of the sample mean.

The Monte Carlo method needs a lot of random sampling to increase its precision (remember the very slow convergence rate). However, the Monte Carlo (MC) method is the only method we have to compute hyper-volumes in hyperspaces, to achieve evenly distributed space filling in large dimensions for experimental designs and where accurate mathematical methods are often intractable or non-existing. Uniformity is guaranteed up to 623 dimensions for the Mersenne Twister (eg. hyperspace of parameters for sensitivity analysis : 100 parameters => 100 dimensions to explore). AI training also requires massive MC sampling far above the 623 dimensions (billions of billions of parmaters create incredibly huge hyperspace)

Compute a confidence interval for 10 and then 40 simulations which estimate PI with 10^9 points. Use parallelism as learned in the 2nd learning situation. Test the 2 generators presented in question 1!

Apppendix: Computing confidence Intervals for a sample of results

Introduction: If X is a simulation result, (X_1 , ... , X_n) is the set obtained with n independent replicates (experiments) of this stochastic simulation. This means that each independent stochastic simulation experiment is run (under the same conditions) but with a different (and independent) random stream. Usually the computing of confidence intervals is achieved on the observed arithmetical means.

$$\bar{X}(n) = \frac{\sum_{i=1}^{n} Xi}{n}$$

The computing of a confidence interval is simple when we consider that the X_i variables (simulation results) have identical independent Gaussian distributions.

Principle: The confidence interval is centered on the arithmetical mean, so we just compute what is called the confidence radius (error margin). Here is the theoretical statistical hypothesis that is used to obtain this radius. If X_i have identical independent Gaussian distributions with a theoretical mean μ and a variance of σ^2, then the following random variable:

$T(n) = \dfrac{\overline{X}(n) - \mu}{\sqrt{S^2(n)/n}}$ is distributed according to a Student law with *n-1* degrees of liberty.

$$S^2(n) = \frac{\sum_{i=1}^{n} \left[X_i - \overline{X}(n)\right]^2}{n-1}$$

$S^2(n)$ is an estimate without bias of the σ^2 variance. Since we don't know the theoretical standard deviation σ, we estimate the variance with the results we have and thus use this approximation. We compute $S^2(n)$ and use it in the following formula below to obtain the confidence radius R (error margin) at the 1-α level :

$$R = t_{n-1,1-\alpha/2} \times \sqrt{\frac{S^2(n)}{n}}$$ (with $\frac{S}{\sqrt{n}}$ being the estimated standard error of the mean)

William Sealy Gosset introduced the Student to correct the fact that we only use an estimate of σ and not its true value. The table below gives the values $t_{n-1,1-\alpha/2}$ for a student law with $\alpha = 0.05$ ($t_{0,975}$).

If $\alpha = 0.05$, the confidence interval is said at 95%. The computing of R gives the following interval [$\bar{X}$- *R*, $\bar{X}$ + *R*] with a 1-α confidence (95%). Under the usual independent-sampling assumptions, this allows you to construct an approximate 95% confidence interval.

Table 1: Values of $t_{0,975}$ ($t_{n-1,1-\alpha/2}$) - Student law starting with $\alpha = 0.05$ depending on *n* experiments.

1≤n≤10	$t_{n-1,1-\alpha/2}$	11≤n≤20	$t_{n-1,1-\alpha/2}$	21≤n≤30	$t_{n-1,1-\alpha/2}$	n>30	$t_{n-1,1-\alpha/2}$
1	**12.706**	**11**	**2.201**	**21**	**2.080**	**40**	**2.021**
2	**4.303**	**12**	**2.179**	**22**	**2.074**	**80**	**2.000**
3	**3.182**	**13**	**2.160**	**23**	**2.069**	**120**	**1.980**
4	**2.776**	**14**	**2.145**	**24**	**2.064**	**+∞**	**1.960**
5	**2.571**	**15**	**2.131**	**25**	**2.060**		
6	**2.447**	**16**	**2.120**	**26**	**2.056**		
7	**2.365**	**17**	**2.110**	**27**	**2.052**		
8	**2.308**	**18**	**2.101**	**28**	**2.048**		
9	**2.262**	**19**	**2.093**	**29**	**2.045**		
10	**2.228**	**20**	**2.086**	**30**	**2.042**		

The Central Limit Theorem (CLT) says that for non-normal data, the distribution of the sample means has an approximate normal distribution, no matter what the distribution of the original data looks like, as long as the sample size is large enough (usually at least 30) and all samples have the same size. This is true not only for the sample mean but also for other sample statistics (find the table for α = 0.01).